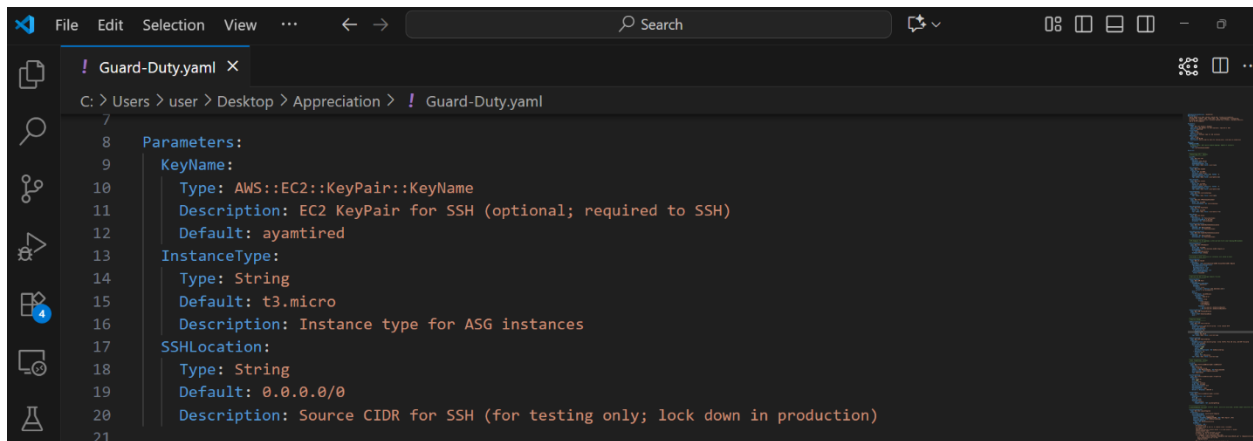


OFFENSE-TO-DEFENSE

(THREAT DETECTION USING GUARD-DUTY)

PREPARATION

Created and deployed a CloudFormation template for reproducible adversary emulation in AWS, simulating an end-to-end attack → detect → remediate scenario.

A screenshot of a code editor window with a dark theme. The title bar shows 'Guard-Duty.yaml' with a close button. The file path is 'C:\> Users > user > Desktop > Appreciation > ! Guard-Duty.yaml'. The code is a CloudFormation template with the following parameters:

```
7
8 Parameters:
9   KeyName:
10     Type: AWS::EC2::KeyPair::KeyName
11     Description: EC2 KeyPair for SSH (optional; required to SSH)
12     Default: ayamtired
13   InstanceType:
14     Type: String
15     Default: t3.micro
16     Description: Instance type for ASG instances
17   SSHLocation:
18     Type: String
19     Default: 0.0.0.0/0
20     Description: Source CIDR for SSH (for testing only; lock down in production)
21
```

The lab involved spinning up Juice Shop, simulating credential theft, validating detection and hardening controls.

Template Preparation

From the CloudFormation Stacks page, selected Create Stack, with new resources (standard).

On the Create Stack page, under Specify a template, chose upload a template file, selected the yaml file from local storage, and confirmed the file was uploaded.

Create stack

Prerequisite - Prepare template

You can also create a template by scanning your existing resources in the [IaC generator](#).

Prepare template

Every stack is based on a template. A template is a JSON or YAML file that contains configuration information about the AWS resources you want to include in the stack.

☒ **Choose an existing template**
Upload or choose an existing template.

☐ **Build from Infrastructure Composer**
Create a template using a visual builder.

Specify template Info

This [GitHub repository](#) contains sample CloudFormation templates that can help you get started on new infrastructure projects. [Learn more](#)

Template source

Selecting a template generates an Amazon S3 URL, where it will be stored. A template is a JSON or YAML file that describes your stack's resources and properties.

☐ **Amazon S3 URL**
Provide an Amazon S3 URL to your template.

☒ **Upload a template file**
Upload your template directly to the console.

☐ **Sync from Git**
Sync a template from your Git repository.

Upload a template file

Guard-Duty.yaml

JSON or YAML formatted file

Verified the template selection and proceeded by clicking next.

Stack Details

On the Specify Stack Details page, entered a stack name.

Left parameters at their default values and selected next to complete it.

Next, On the Configure Stack Options page, checked the box acknowledging that CloudFormation might create IAM resources.

Chose Next to continue.

Review and Deployment

Reviewed the configuration on the Review and create page.

Submitted the stack for deployment.

The stack creation process took about 10 minutes to complete.

After successful creation, reviewed the Outputs tab.

Recorded the values for JuiceShopURL and TheSecureBucket for later use.

Step 1: Specify template

Prerequisite - Prepare template

Template

Template is ready

Template

Template URL

<https://s3.eu-north-1.amazonaws.com/cf-templates-cldo3t4rstcn-eu-north-1/2025-09-25T102523.705Z0ca-Guard-Duty.yaml>

Stack description

Deploy OWASP Juice Shop (Docker) behind ALB, fronted by CloudFront. Includes VPC, subnets, IGW, route table, ALB, LaunchTemplate, S3 bucket for "sensitive.txt" (instance uploads file on boot), GuardDuty detector, and an S3 VPC endpoint.

The screenshot shows the AWS CloudFormation console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and account information (Europe (Stockholm), Account ID: 1630-8893-3322, UltimateIAM). Below the navigation bar, a blue banner announces a new Control Catalog integration for AWS CloudFormation Hooks. The main content area is titled 'Stacks (1)' and shows a list of stacks. The 'Ro-GuardDuty2' stack is highlighted, showing its status as 'CREATE_COMPLETE'. To the right, the 'Stack info' tab is selected, displaying the stack's ID, description, and various status fields.

Stacks (1)

Filter status: Active

View nested

Stacks

- Ro-GuardDuty2
2025-09-25 14:55:59 UTC+0000
CREATE_COMPLETE

Ro-GuardDuty2

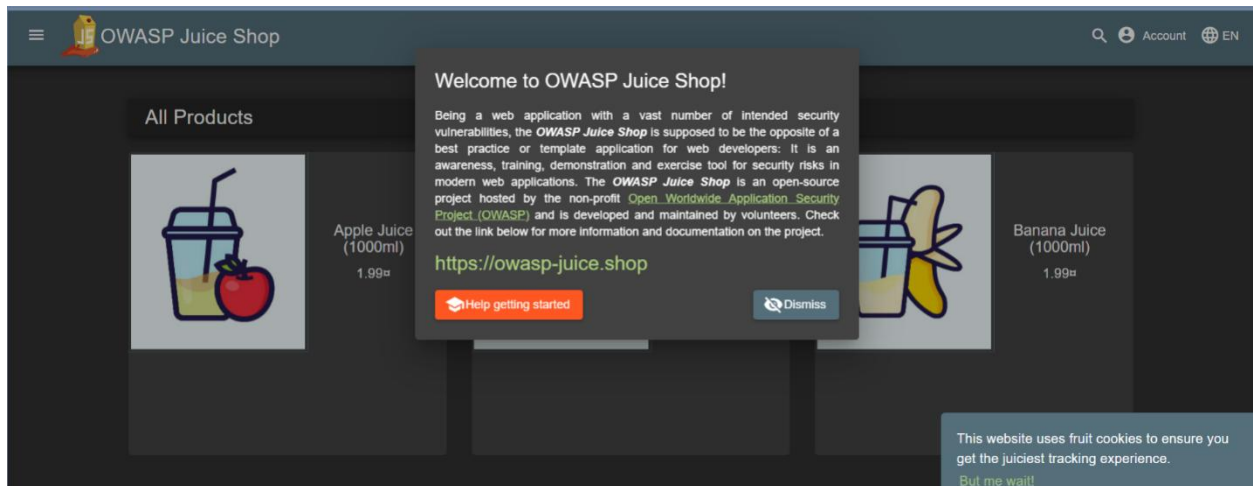
Actions: Delete, Update stack, Stack actions, Create stack

Stack info | Events | Resources | Outputs | Parameters | Template | Change sets | Git sync

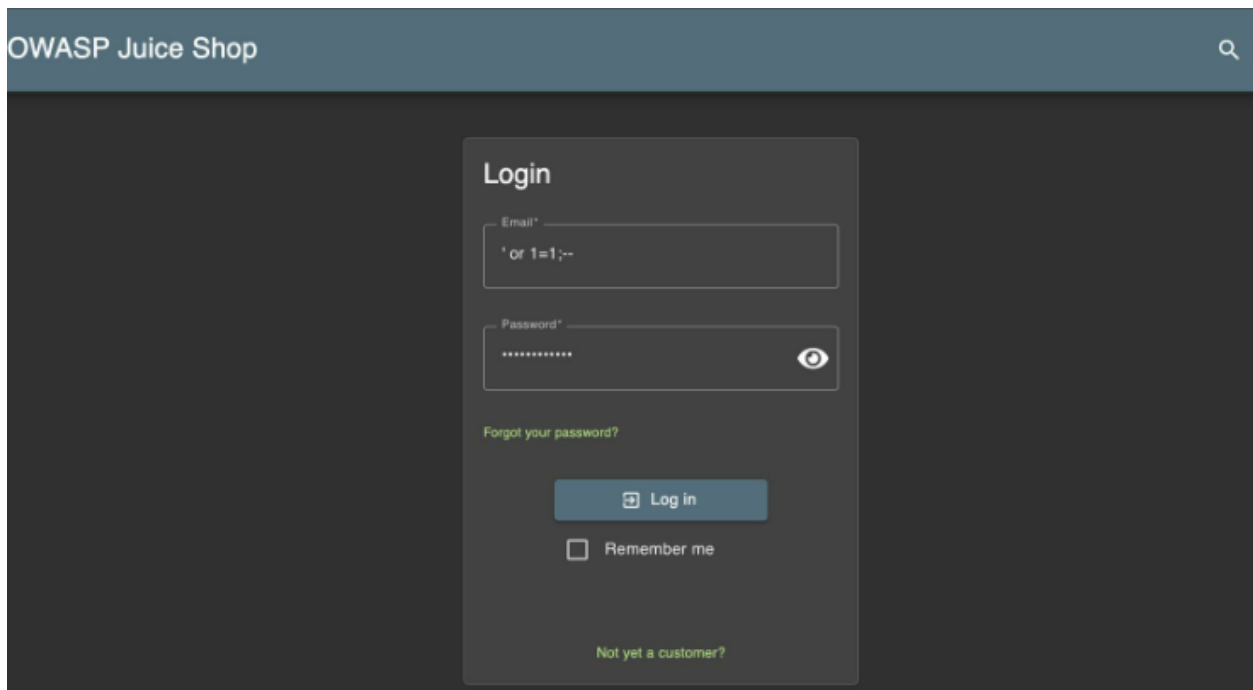
Overview

Stack ID arn:aws:cloudformation:eu-north-1:163088933322:stack/Ro-GuardDuty2/bf173c00-9a1f-11f0-83ff-0ea265cef27d	Description Deploy OWASP Juice Shop (Docker) behind ALB, fronted by CloudFront. Includes VPC, subnets, IGW, route table, ALB, LaunchTemplate + AutoScaling, S3 bucket for "sensitive.txt" (instance uploads file on boot), GuardDuty detector, and an S3 VPC endpoint.
Status CREATE_COMPLETE	Detailed status -
Status reason -	Root stack -
Parent stack -	Created time 2025-09-25 14:55:59 UTC+0000

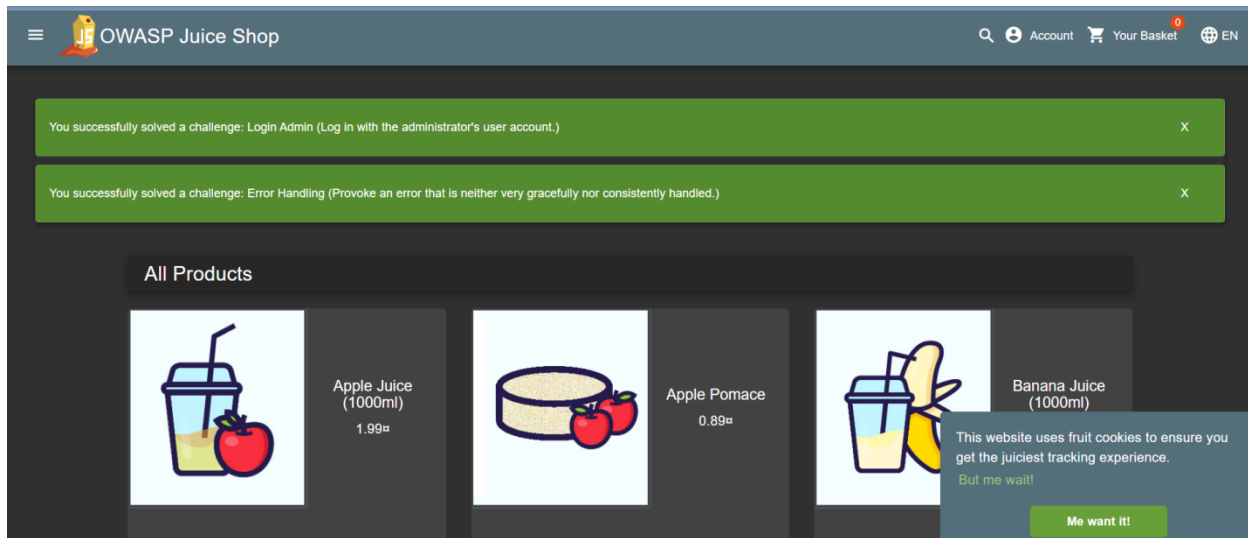
Next, accessed the deployed Juice Shop from the URL. The OWASP Juice Shop homepage loads. Dismissed the initial welcome dialog, and accepted the cookie notice when prompted.



Authenticated as an administrator using a login bypass technique.



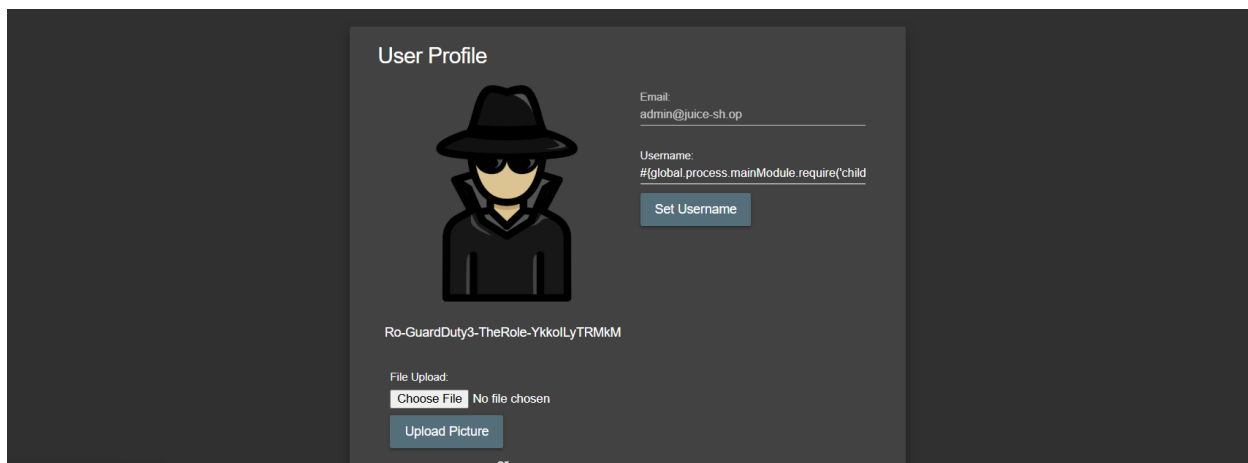
After submitting the credentials, confirmed that the sign-in succeeded.



GETTING AWS CREDENTIALS

From the admin account, open Account → User Profile. On the profile page, set the username using an input that triggers server-side command execution.

Verified that the page shows the command output beneath the username avatar, indicating the process/user identity was printed to the page.



The public directory aws-creds.json was then created. I accessed it by using the following URL attached to the Juice Shop homepage Url:

<JuiceShopURL>/aws-creds.json

The contents of the JSON file should look like this (the values will most likely be different): I copied the details for the AccessKeyId, SecretAccessKey, and Token. The three elements that make up the authentication information, AccessKeyId, SecretAccessKey, and Token are valid until the Expiration date and time. In Lehmann's terms, the attacker has now been successful in obtaining the web application's AWS authentication credentials.

```
d2mcpctmqppg.cloudfront.net/aws-creds.json

Sharing https://zenn.dev to meet.google.com

Stop sharing Share this tab instead

pretty print

{
  "Code": "Success",
  "LastUpdated": "2025-09-26T17:17:42Z",
  "Type": "AWS-IAM",
  "AccessKeyId": "ASIASEG3UHFKTUWUUV",
  "SecretAccessKey": "Hj9wLKjAqKIDPs13o80I21PawWQI0wi0dcM6/Lc",
  "Token": "IQo3b3p221uXZVJEAKuCV1lW5ccnRoL TE1SDRGaIEA4denn1kZkDvIeNlRgUvL YONSgAtoJruUgpr1NPwgEQDk13haF1NkwpLEJTD8LjQuXh41eQ/1K1HPH1QJOG8XC+FBQ1S/////////BBEAAuDE2MaA40KzN2hYALIPJhWuJg/qjce+PqWpKfFrbwh1X4pdKUR/wUu o1BhVNPVMGmQgqEBP1Nt9FD715163AgYkmf4CupXY/nAMBFkDEs9s1RTechut+nugq/hvds1mWZvkoJ8/cw8T9HBL4Sikmmuk712jYIG039598rs9TuCZYSaAZ3QaeS1UoaYc:FHVpvtV01/UrkBBAYsDcLiiaNGWmPdeazv0xdL8ZEK2LK63e7YX6R8DuyIrrmTEhh6E1CG4 bAahciQwTQmZjdcb75eCmZGdwe1X3jKevGmP06zQOYLImR01M4r13M/pHYAuEDps5z03f0M3zDgsUZEUDvPB4e1fotAkpFpw6ogsX2YmwsbFPR40t9yV65rnbAupt6oEwh/TmM+Q8jAShap8jXnItuay7uMtAbYQZBS1xf1N660954undurPYWATm3VCFMAA015B03RqM7FftzXo r7unH8+P1um1MAuKzE72vIPna9tRt14B85n8+YF56g8j5rTsXN0sU7180m559kdyG1g99t2tc+nK/ybdc1GhQomM0p/3121f7cbvn16WcY1kgo/E2mu8k178aAurGkBAAYs0GMS+XZWe1vRMA3FxmR80tF1pTt4281tkyfo34VWekM7eMSUcnCDF6mqPCatgoqrueppETpFKi0 x0u17/7Z75b1lGwb1Yjzg64acCXZ/51qrw0tQ/X211X9XS72t0pUaM8efav67gU0h+0TbnVSbub2aMNHfCy3hQ657u5jyC1m611CrBjdFBUmE8nAnKX9Kj9BuZ+0F18QQ0ouToK26u1/h3AuyYkUJT10orToGk1sD6BBWAP/F8jEa//Xr2T3efg1Uw153phTrIcy5NTgSU12FC/VB cCr1YER1Upk8k1p8bzgY6s4MEWkcs7h8tY11tqDy0A8K1Qo7qG0AAVakwo/ftdoqDeCbm82HvEP0L73Th81Hbult14u36/YTh64exAgghs11+1TFPLy3H85g2jEBp52EdpPCNfPKk1CQ+jxShc1ag1CLSo)c5111d933+/z0u59sQzYe7H3ms5KfG8+j74GPa6mwc21W1ngyYH8 6AMpeKtdk3fH8aYud1fshYfzHLT67/1ghysyEar1sVCsA==",
  "Expiration": "2025-09-26T23:51:58Z"
}
```

RETRIEVING FILES IN

ro-guardduty3-thesesecurebucket-uywrxrq3buuj [Info](#)

Objects Properties Permissions Metrics Management Access Points

Objects (1) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	secret-information.txt	txt	September 26, 2025, 17:19:14 (UTC+00:00)	78.0 B	Standard

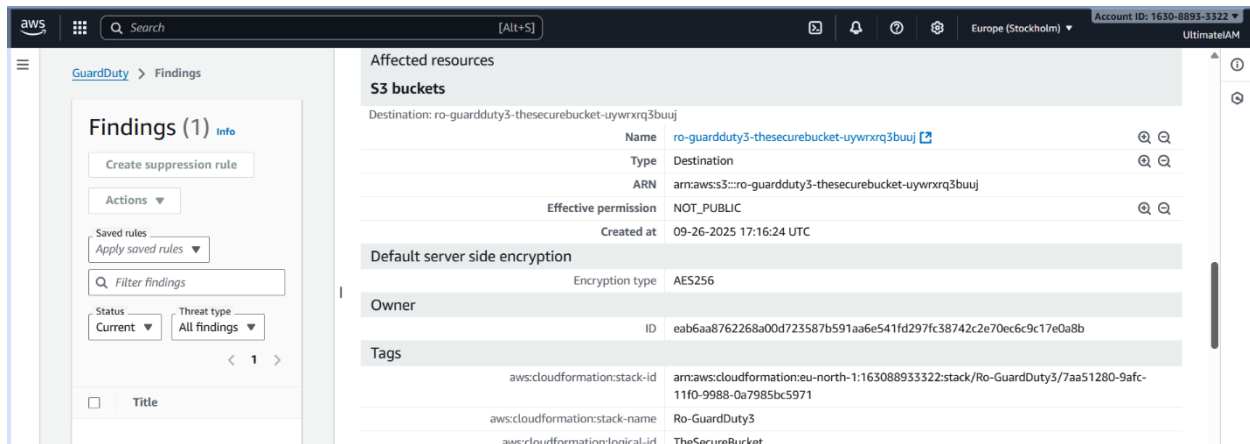
Open a terminal prompt on the machine for the lab, go through the AWS Configure process and then set the AWS credentials obtained as the environment variables

Next, set an environment variable for the S3 bucket name using the value shown in the CloudFormation stack outputs (TheSecureBucket value):

set AWS_SECURE_BUCKET=<TheSecureBucket value>

Next, list the objects in the bucket with:

GuardDuty monitors communication packet flows documented in VPC Flow Logs and AWS actions documented in CloudTrail, and it identifies any unusual or suspicious activity. In this case, it was discovered that session credentials meant to be used inside an EC2 instance were being utilized from an external IP address or a different AWS account.



DETECTING MISCONFIGURATIONS WITH CONFIG AND SECURITY HUB

Enable AWS Config. In the AWS Management Console, open Config. Select Start Now.

On the Settings page, keep the defaults and click Next. On the Rules page, leave the defaults and click Next. On the Review page, choose Confirm.

Enable Security Hub & Best Practices

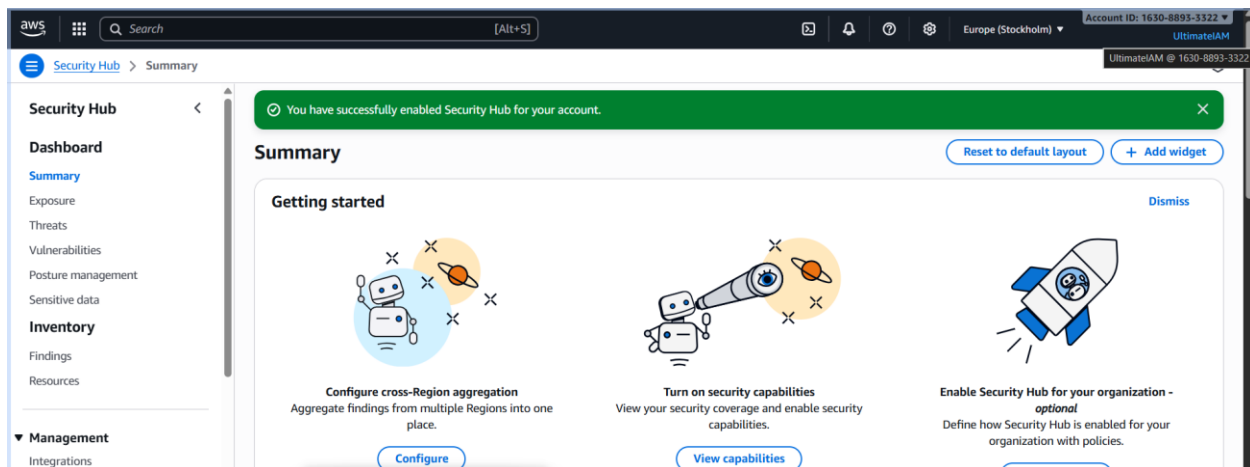
In the console, open Security Hub and click Start using.

On the Enable Security Hub page, choose Enable.

After enabling, you'll see a confirmation message. Click Go to Security Hub.

From the navigation pane, select Detection Engine → Security Hub CSPM.

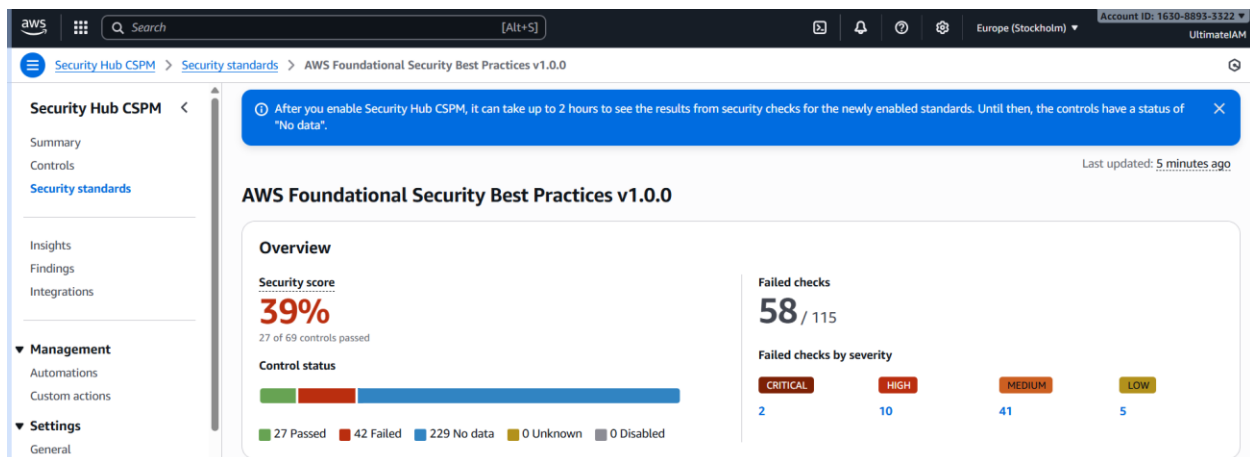
On the CSPM homepage, select Enable AWS Foundational Security Best Practices v1.0.0 and confirm.



If Security Hub CSPM was already enabled, go to Security Standards and enable AWS Foundational Security Best Practices v1.0.0 manually.

Check Results

Once the checks run, the Security Hub CSPM dashboard will show a security score and highlight failed controls.



Open the AWS Foundational Security Best Practices v1.0.0 report to view details.

Use the search filter control = FAILED to display only failed controls.

Switch to ID-level view by clicking the ID column.

Confirm that one detected issue matches the root cause of the attack:

EC2.8: EC2 instances should enforce Instance Metadata Service Version 2 (IMDSv2).

aws

Search

[Alt+S]

Europe (Stockholm)

Account ID: 1630-8893-3322

UltimateIAM

Security Hub CSPM

Security standards

AWS Foundational Security Best Practices v1.0.0

EC2.8

Security Hub CSPM

Summary

Controls

Security standards

Insights

Findings

Integrations

Management

Automations

Custom actions

Settings

General

After you enable Security Hub CSPM, it can take up to 2 hours to see the results from security checks for the newly enabled standards. Until then, the controls have a status of "No data".

Last updated: 11 minutes ago

EC2 instances should use Instance Metadata Service Version 2 (IMDSv2)

[EC2.8] This control checks whether your Amazon Elastic Compute Cloud (Amazon EC2) instance metadata version is configured with Instance Metadata Service Version 2 (IMDSv2). The control passes if HttpTokens is set to required for IMDSv2. The control fails if HttpTokens is set to optional. [Remediation instructions](#)

Overview

Security Hub CSPM determines control status based on control findings in the last 24 hours. Severity signifies the importance that Security Hub CSPM has assigned to a control.

Control status

Failed (Updated 11 minutes ago)

Severity

High

Compliance status

0 Passed

1 Failed

0 Warning

0 Not Available